

CoDesign (Hardware/ Software)
Processeur SoftCore Microblaze de Xilinx
(kit Basys3 de Digilent)

Logiciels :

- Vivado & SDK (v19.1)
- IP Integrator pour la conception du système sur puce programmable

Plateforme :

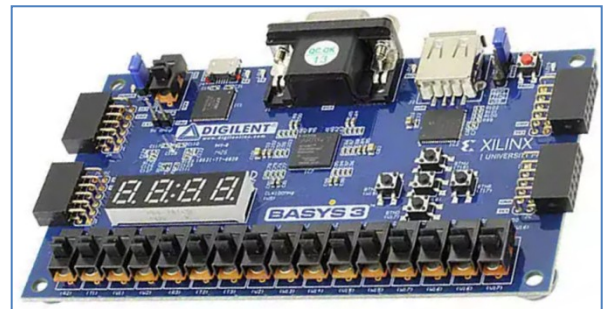
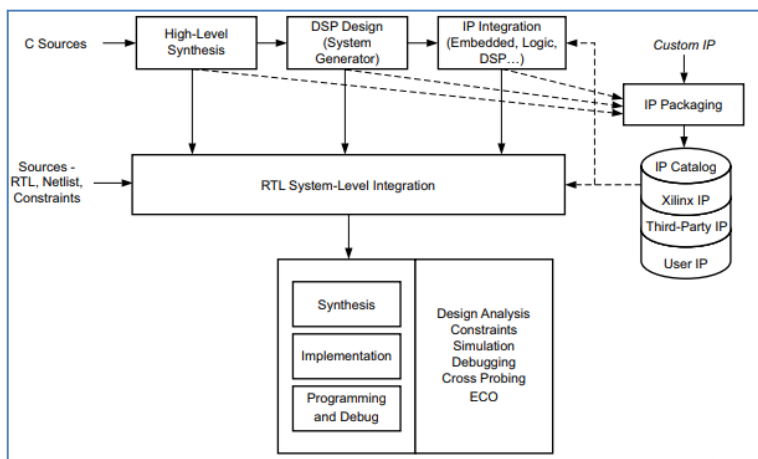
- Kit Basys3 de Digilent

FPGA : Xilinx **Artix-7**, package **XC7A35T-1CPG236C**

- 5200 slices, 33280 logic cells
- BRAM 1800 Kbits
- horloge > 450 MHz

Objectif du tutoriel :

L'objectif de ce tutoriel est de découvrir l'approche **SoPC (System On Programmable Chip)** de Xilinx. La méthodologie de conception consiste à mener conjointement le développement du système complet en prenant en compte les deux aspects : matériel (hardware) et logiciel (software). Les applications de ce tutoriel ciblent une plateforme FPGA (*Basys3* de *Digilent*) mais aussi une plateforme de type SOC-FPGA (*Zybo* de *Digilent*). Les SOC-FPGA intègrent, sur la même puce, un cœur FPGA et un processeur câblé (hardcore). La figure ci-dessous montre le flot de conception hard/soft et une photo de la carte de développement utilisée.



Prise en main de la chaine de conception pour Microblaze (Vivado + SDK)

Cette étape va nous permettre de découvrir la suite logicielle *Vivado* de Xilinx. Nous allons l'utiliser pour concevoir et tester une architecture autour du processeur **softcore Microblaze**. Le kit de développement que nous allons utiliser, *Basys3* de *Digilent*, est construit autour d'un FPGA. Il s'agit d'un des derniers kits de développement de *Digilent*. L'objectif de ce tutoriel est de vous faire découvrir étape par étape la conception d'un système **Microblaze** en utilisant l'outil **IP Integrator** de **Vivado**. La finalité de ce tutoriel est de :

- Maîtriser la conception hardware d'un système **Microblaze** dans *Vivado*.
- Développer pour le système **Microblaze** ainsi réalié un projet software en C en utilisant l'environnement de développement logiciel Xilinx Vivado SDK.
- Utiliser des outils annexes pour communiquer avec **Microblaze** (console SDK/Tera Term).

▪ Caractéristiques du kit Basys3

Voici les ressources matérielles du kit Basys3 fournies par Digilent :

- [Xilinx Artix-7 FPGA : XC7A35T-1CPG236C](#)
- 33,280 logic cells in 5200 slices (each slice contains four 6-input LUTs and 8 flip-flops)
- 1,800 Kbits of fast block RAM
- Five clock management tiles, each with a phase-locked loop (PLL)
- 90 DSP slices
- Internal clock speeds exceeding 450 MHz
- On-chip analog-to-digital converter (XADC)
- Digilent USB-JTAG port for FPGA programming and communication
- Micro-B USB cable not included.
- Serial Flash
- USB-UART Bridge
- 12-bit VGA output
- USB HID Host for mice, keyboards and memory sticks
- 16 user switches
- 16 user LEDs
- 5 user pushbuttons
- 4-digit 7-segment display
- 4 Pmod ports: 3 Standard 12-pin Pmod ports, 1 dual purpose XADC signal / standard

🔗 Pour plus de précisions, consultez la documentation fournie dans le fichier [basys3_rm.pdf](#)

Étapes du tutoriel :

Microblaze est une IP softcore proposé par Xilinx. Il plante un processeur complet dans un FPGA. Dans ce tutoriel, nous allons lui ajouter des éléments supplémentaires en utilisant l'utilitaire IP Integrator (mémoire, UART, PIO, ...)

Les étapes du flot de conception à exécuter dans Vivado

- Ouvrir Vivado et sélectionner le FPGA de la carte Basys3
- Créer un nouveau projet Vivado
- Créer un block de travail dans le nouveau projet
- Ajouter les blocs IP en utilisant l'utilitaire IP Integrator
- Valider et sauvegarder le design
- Créer un toplevel HDL wrapper
- Lancer la synthèse et l'implantation FPGA
- Générer le fichier de configuration "bitstream"
- Exporter l'architecture dans SDK
- Lancer SDK

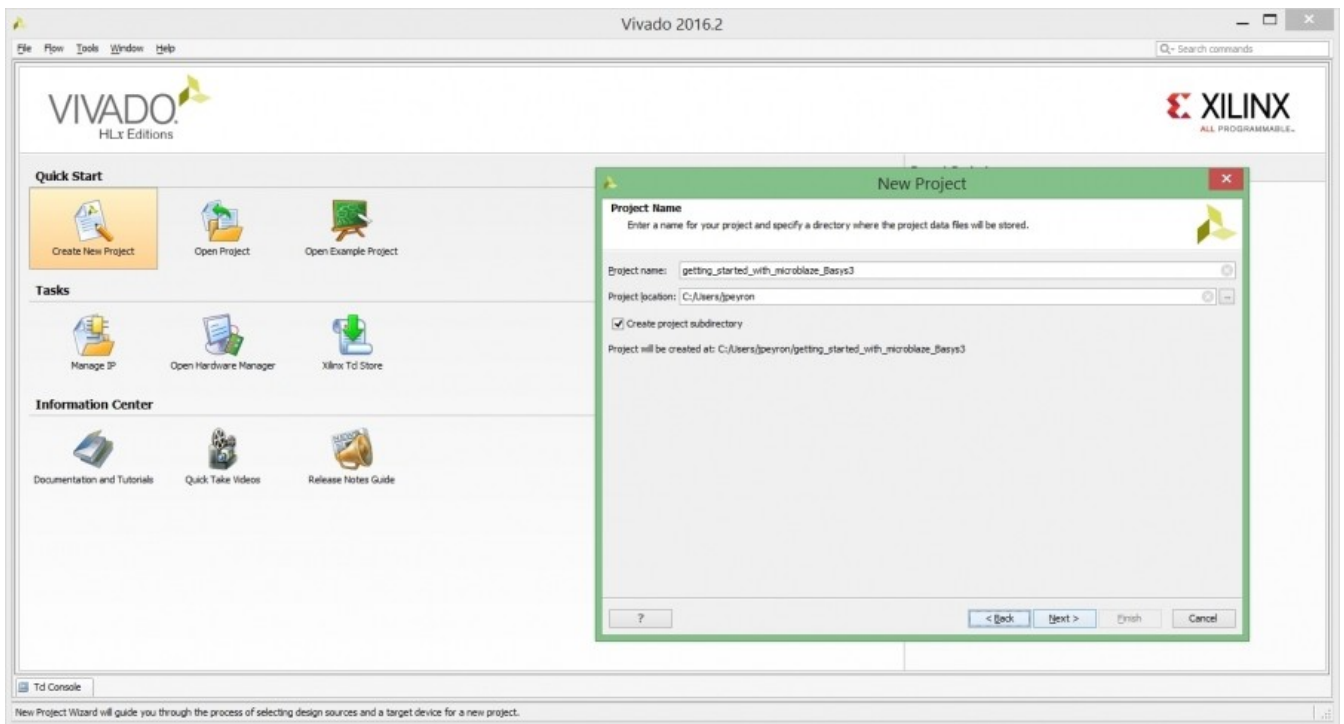
Et dans SDK :

- Créer un nouveau projet software pour développer une application en C
- Programmer le FPGA
- Lancer l'exécution en configurant le port de communication pour l'UART

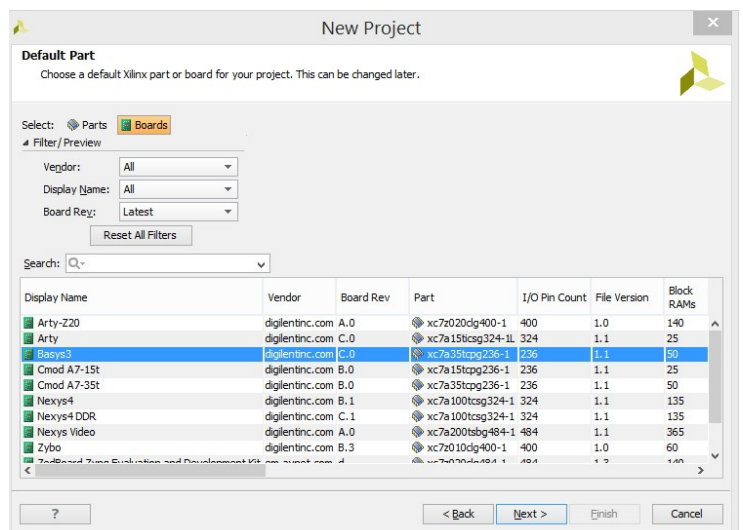
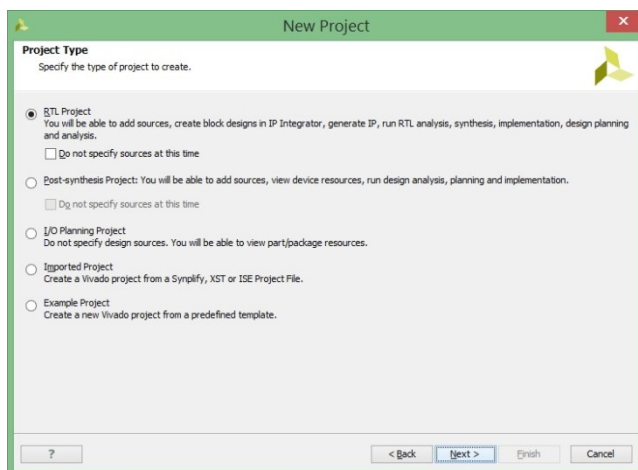
1. Création d'un nouveau projet

Au lancement de Vivado, la fenêtre principale qui s'ouvre vous permet de créer un nouveau projet ou d'ouvrir un projet récemment utilisé.

1.1- Cliquez sur **Create New Project**. Choisissez un nom de projet et un dossier ne contenant pas d'espace et aucun caractère de l'alphabet avec accent. Le caractère *Underscore* "touche 8" représente une bonne alternative à l'espace. L'idéal serait d'avoir un dossier dédié aux projets *Vivado* (exemple : [Projets_Vivado](#)). Nommez votre projet et sélectionnez votre dossier puis cliquez sur **Next**.



1.2- Choisissez le type de projet comme suit **RTL Project** et cliquez sur **Next** jusqu'au choix du FPGA cible.

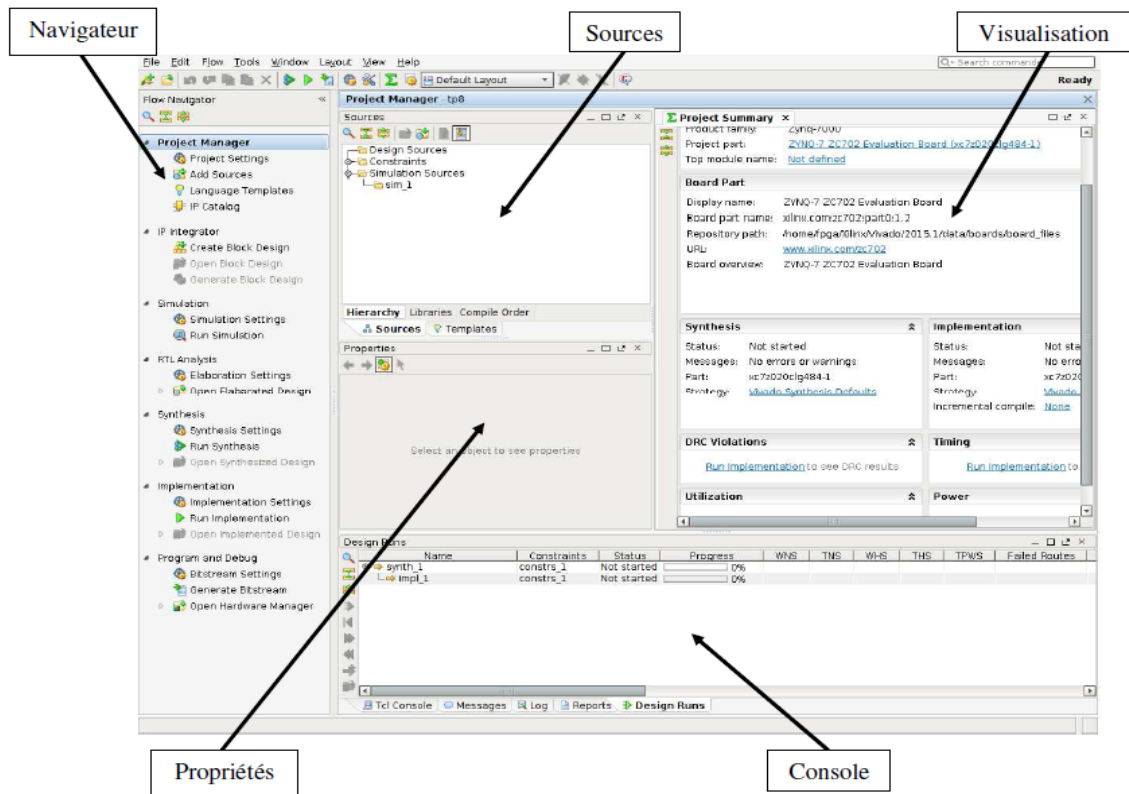


Le plus simple serait de choisir la **board Basys3** (à faire dans un deuxième temps). Pour cette fois-ci et afin que vous puissiez effectuer toutes les étapes, je vous propose de sélectionner le type de FPGA : **Choisissez donc le FPGA XC7A35TCPG236-1** puis cliquez sur **Next**. Un récapitulatif des caractéristiques du nouveau projet s'affiche. Cliquez sur **Finish**.

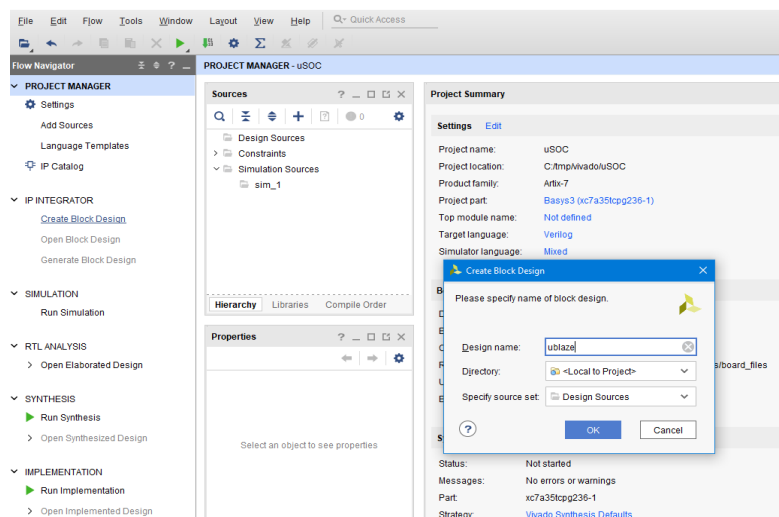
2. Création d'un nouveau bloc

2.1- A la fin de la création du projet, la fenêtre principale qui s'ouvre permet d'effectuer toutes les manipulations nécessaires pour la conception et le test du projet. L'interface du flot de navigation, située à gauche, offre plusieurs options pour créer des blocs hardware, effectuer des simulations, lancer la synthèse et l'implantation physique pour finalement générer le fichier de configuration "bitstream file". La configuration du FPGA peut être faite en utilisant le menu "Hardware Manager".

L'utilitaire *IP Integrator* va nous permettre de concevoir des blocs fonctionnels et/ou d'ajouter des fichiers sources de type RTL.

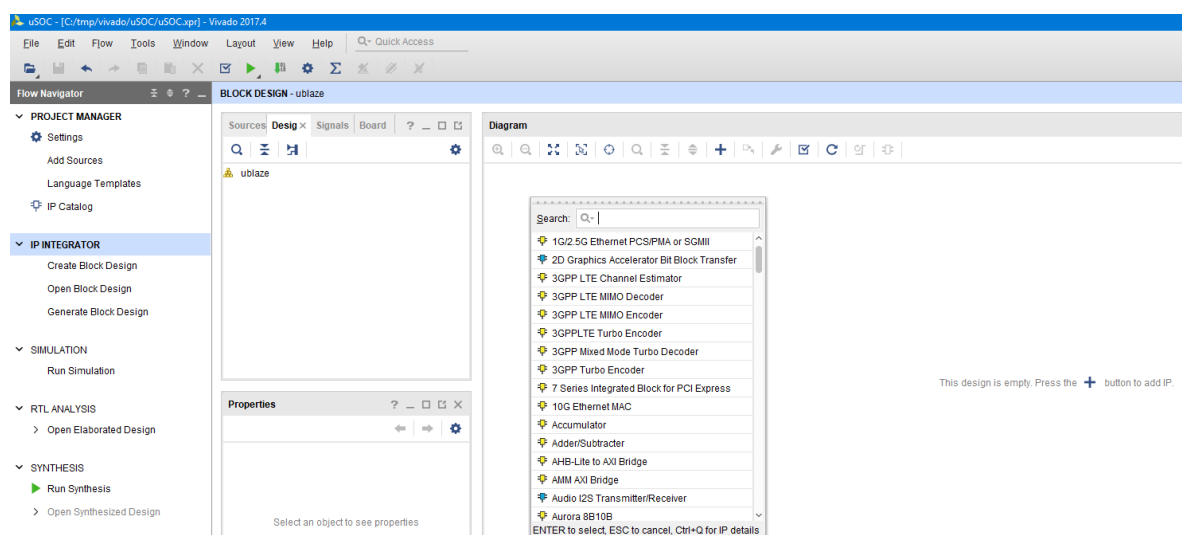


2.2- Sur la gauche, on retrouve donc le flot de navigation "Flow Navigator". Sélectionnez **Create Block Design** dans la catégorie *IP Integrator*. Donnez un nom à votre design en évitant les espaces.



2.3- Un fenêtre de travail vide est allouée pour permettre l'ajout de blocs IP.

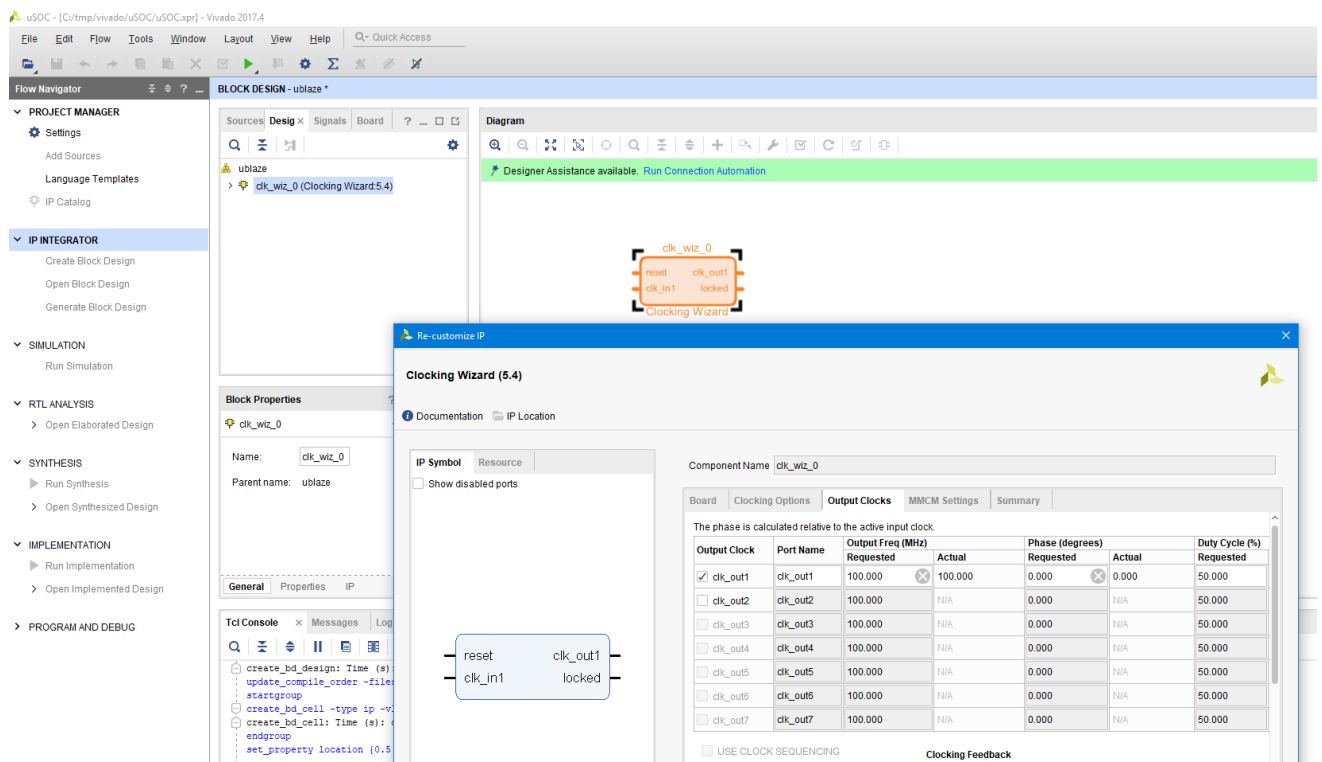
En cliquant sur l'icône  ou  le catalogue d'IP mis à disposition par Xilinx s'affiche.



3. Ajout de l'horloge

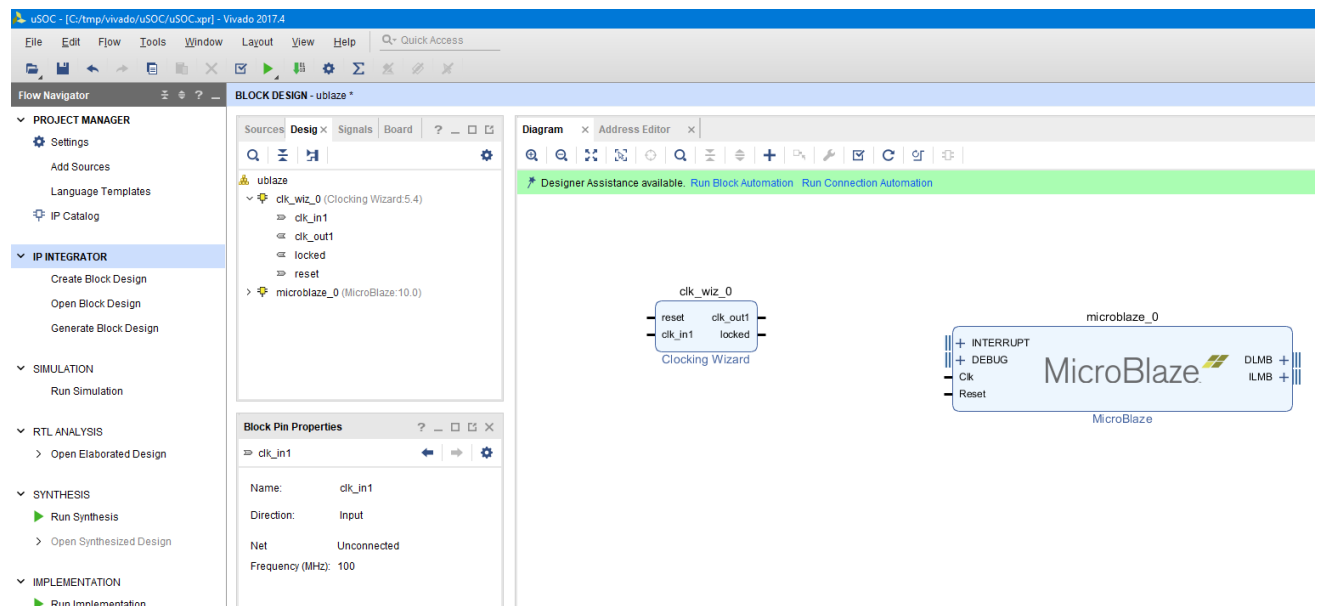
3.1- Ajoutez le bloc d'horloge en sélectionnant dans le catalogue affiché le bloc **"Clocking Wizard"**.

3.2- Double-cliquez sur le bloc ou activez le menu **Customize Block** pour personnaliser l'horloge. Dans l'onglet **Output Clocks** vérifiez que **clk_out** est à "100.000" et le **Reset type** est **Active High**.

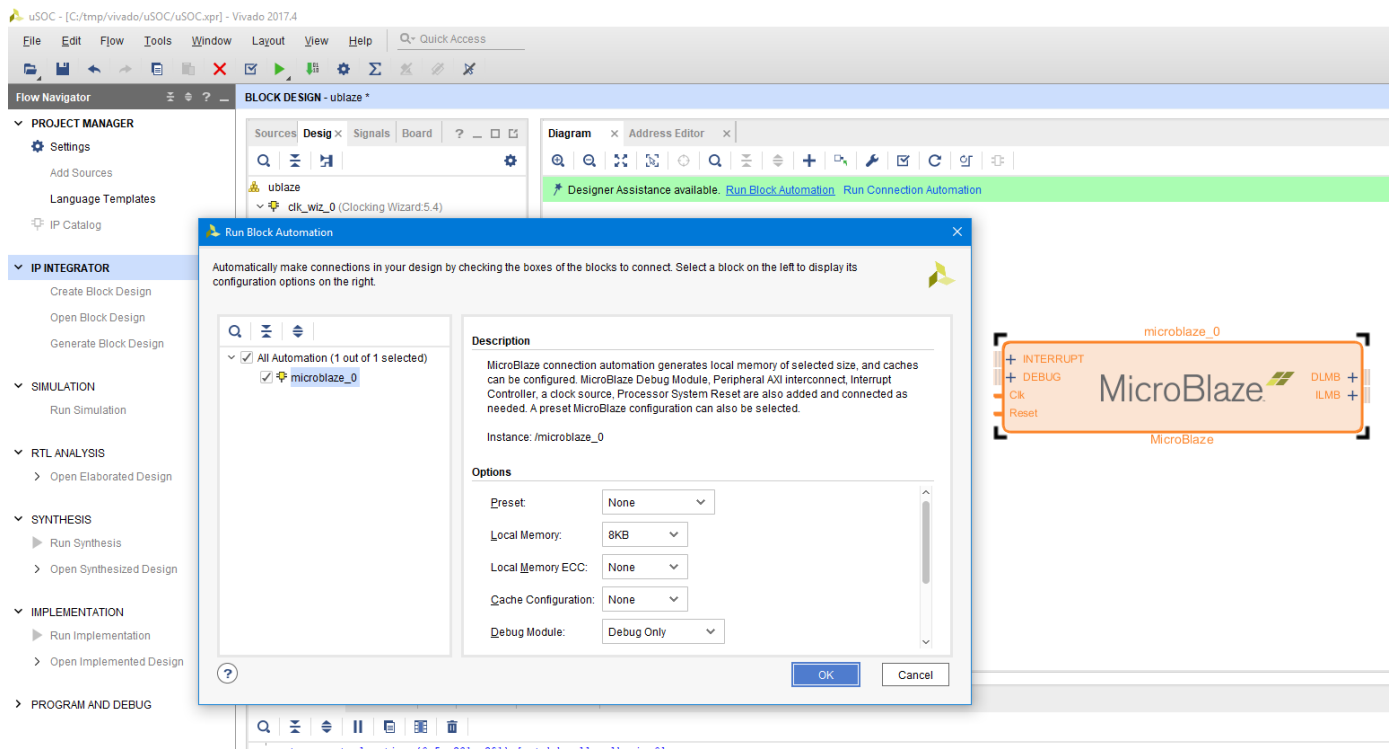


4. Ajout de l'IP Microblaze puis paramétrage

4.1- Ajoutez l'IP Microblaze. Lorsqu'une nouvelle IP est ajoutée, l'utilisateur peut paramétrer les propriétés du bloc soit en cliquant sur le message **Run Block Automation** affiché sur la **bannière verte** en haut de la fenêtre **Diagram** ; soit en double-cliquant sur le bloc lui-même.

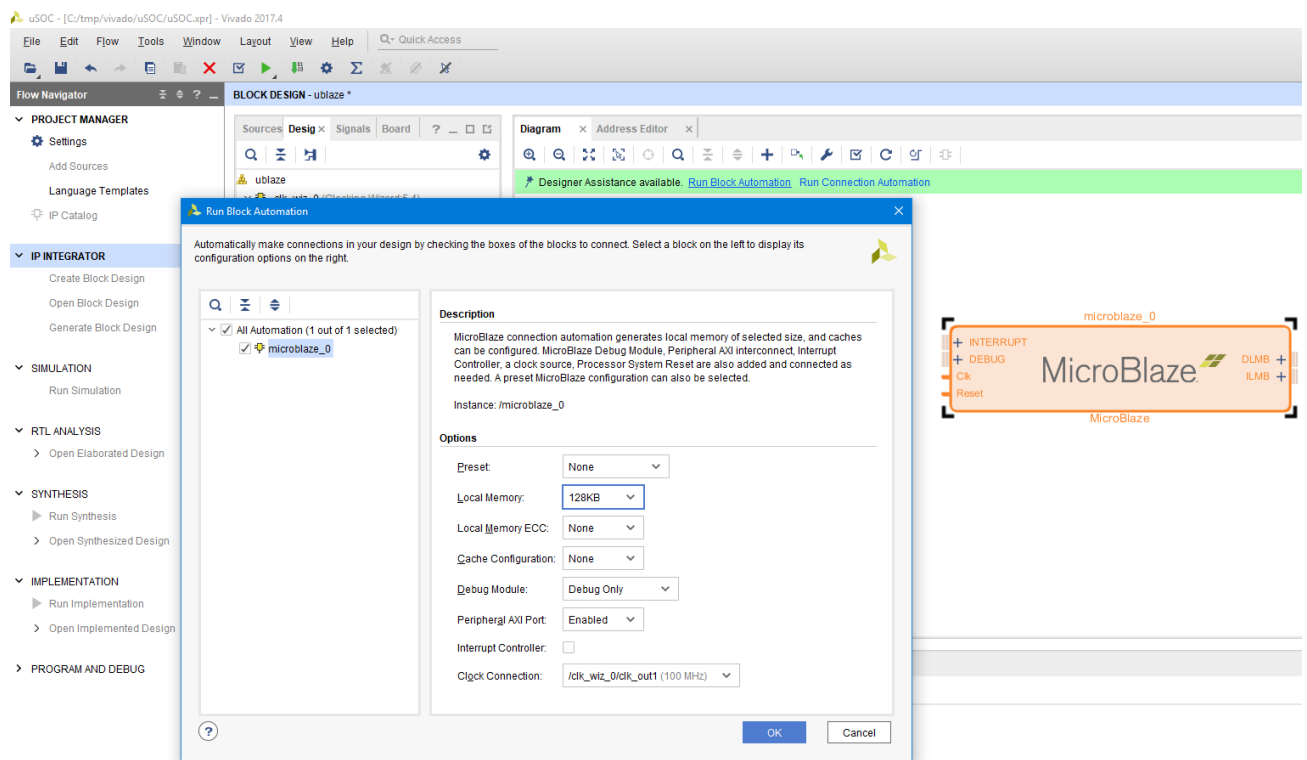


4.2- Sélectionnez le menu **Run Block Automation** et la fenêtre de paramétrage s'affiche avec les valeurs par défaut.

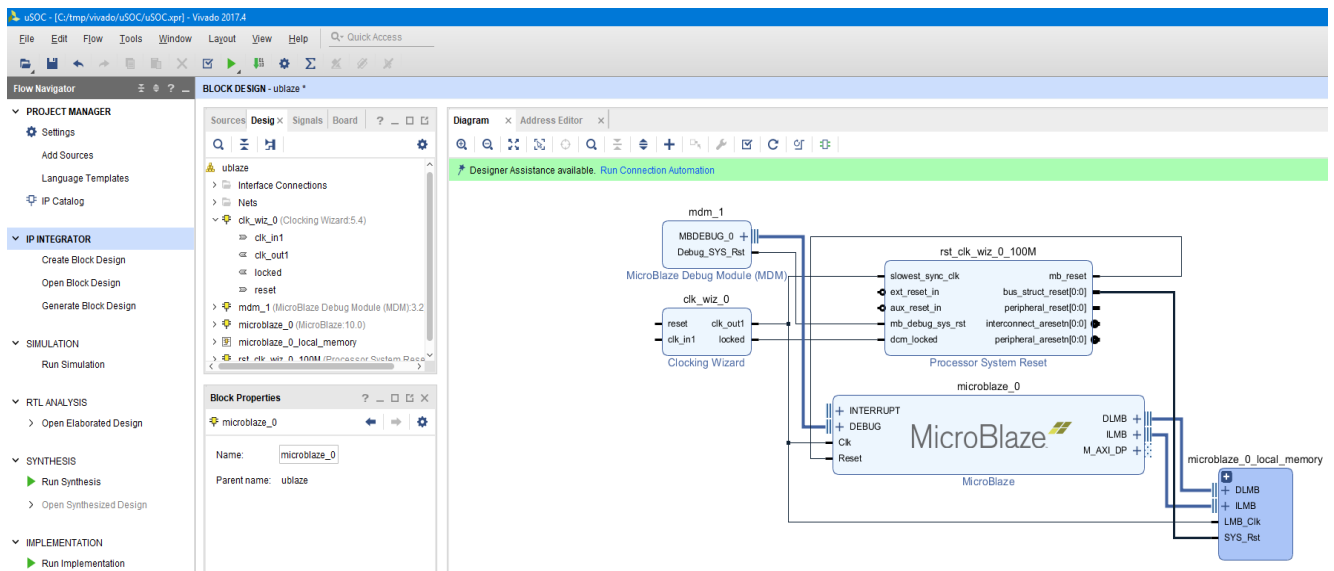


4.3- Changez les propriétés du bloc pour que ça ressemble aux options ci-dessous :

- **Local Memory: 128KB**
- **Local Memory ECC: None**
- **Cache Configuration: None**
- **Debug Module: Debug only**
- **Peripheral AXI Port: Enabled**
- **Interrupt Controller: unchecked**
- **Clock Connection: /clk_wiz0/clk_out1(100 MHz)**

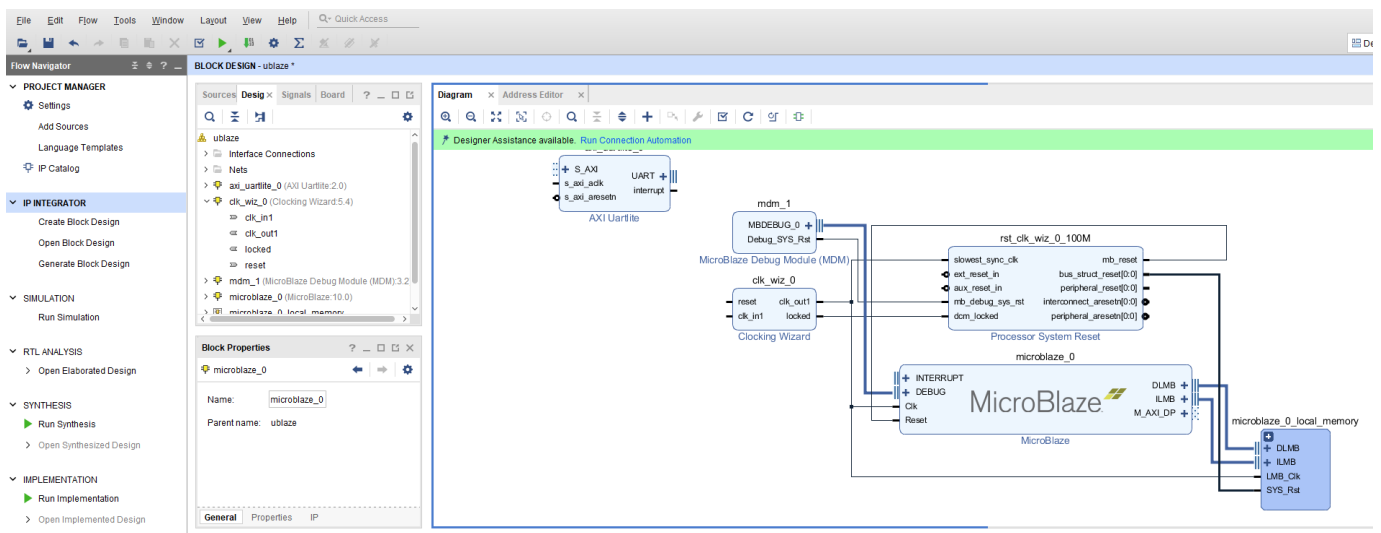


4.4- La sélection du menu **Block Automation** sur Microblaze a pour effet de générer automatiquement des IP additionnels selon les options sélectionnées lors de l'étape de paramétrage précédente.

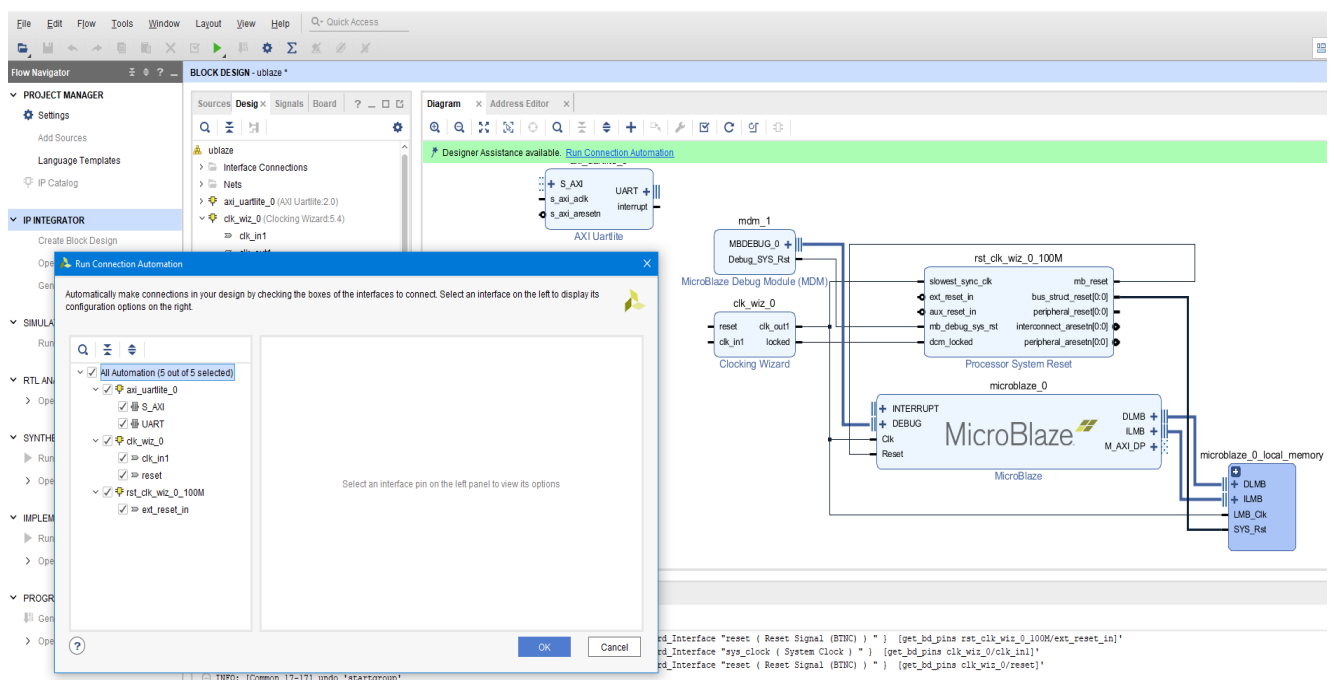


5. Ajout de composants périphériques

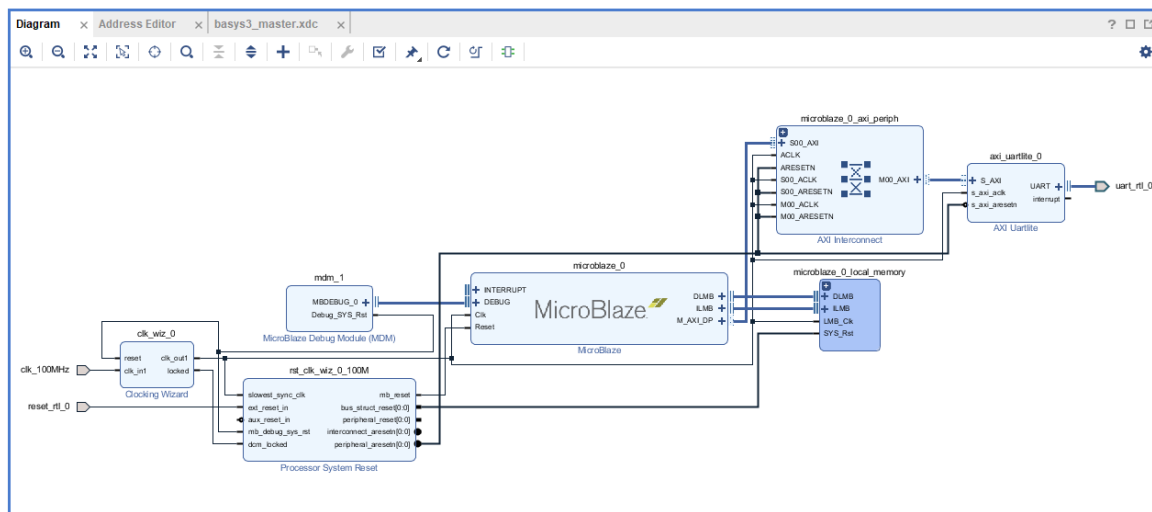
5.1- Ajoutez au design le composant **USB UART** en sélectionnant le bloc **Uartlite** dans le catalogue.



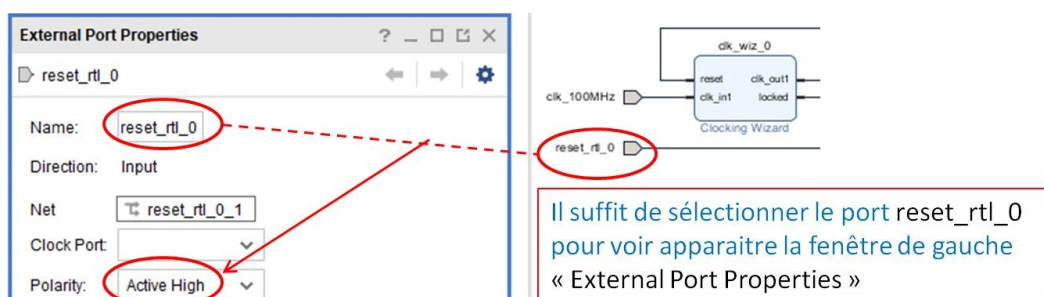
5.2- Cliquez sur **Run Connection Automation** et cochez toutes les options puis appuyez sur **OK**.



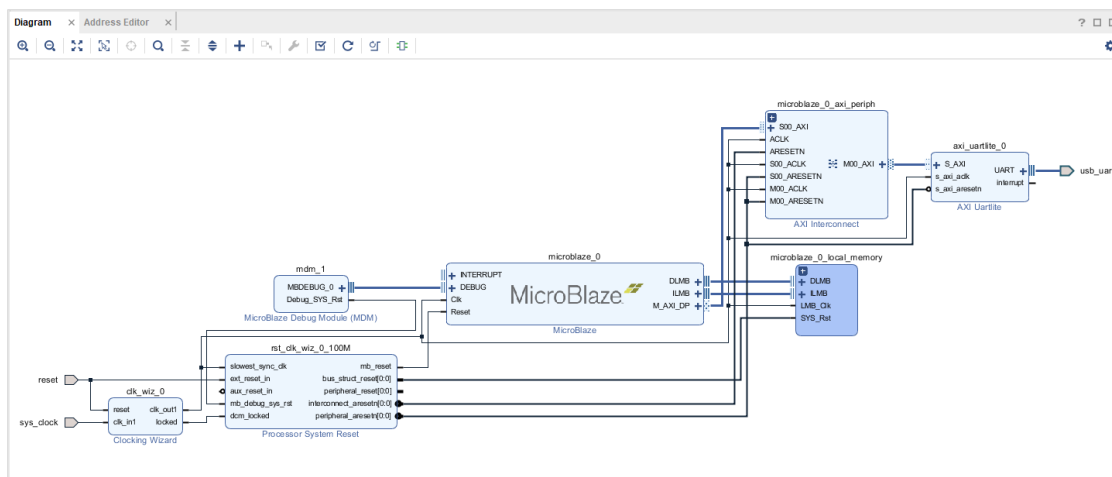
5.3- Sélectionnez  **Validate Design** (ou  selon la version de **Vivado**) pour vérifier d'éventuelles erreurs puis régénérez le schéma en cliquant sur  **Regenerate Layout** (ou ). Le schéma affiché devra ressembler au schéma ci-dessous.



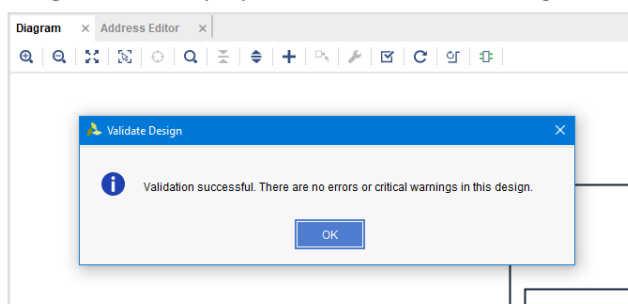
Attention : vérifiez que la propriété « Polarity » du port **reset_rtl_0** est bien fixée à « Active High » (voir ci-après).



Dans le cas où « **board** » a été choisi pour fixer les contraintes des broches, le schéma ressemblerait plutôt à ça :

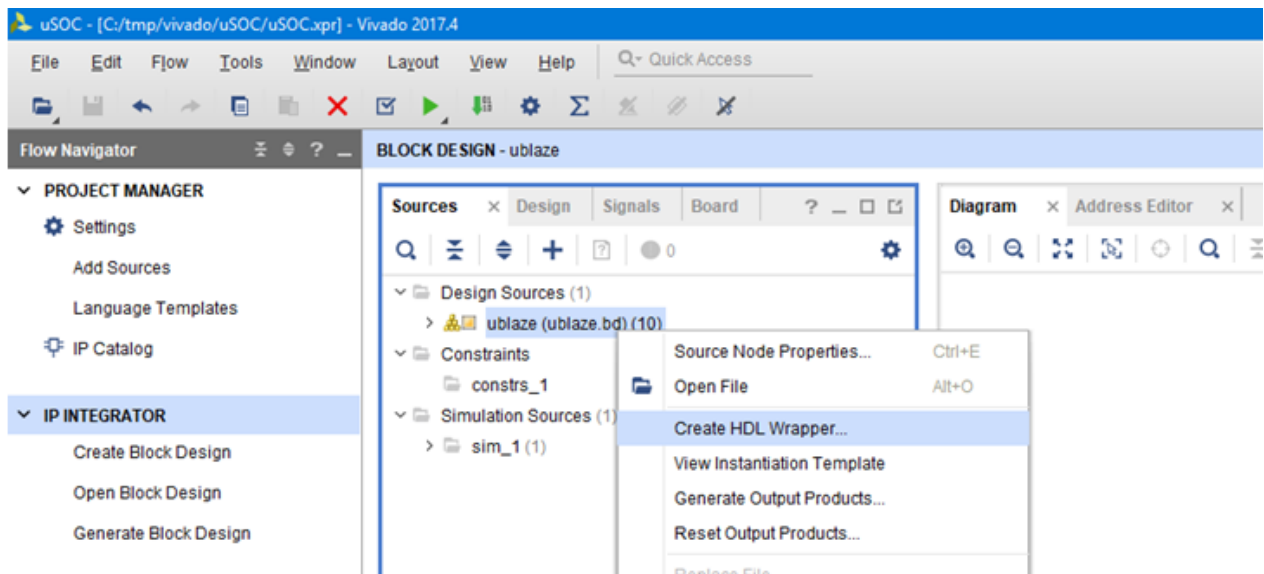


5.4- Sélectionnez  **Validate Design**. Cette étape permet de vérifier le design et les connections.



6. Génération du toplevel du design avec **Create HDL wrapper**

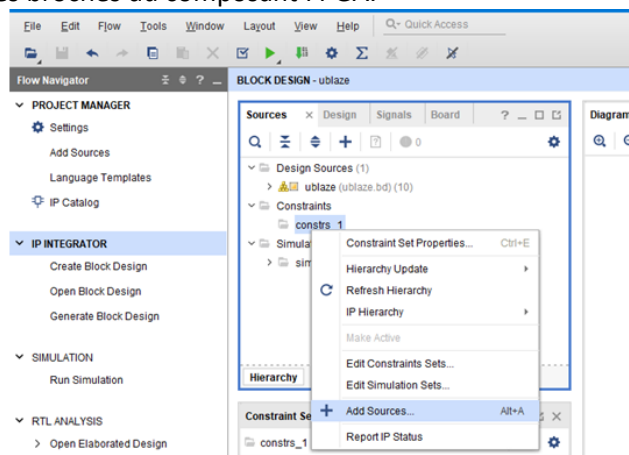
6.1- Après la validation du design nous procédons à la création du module toplevel (qui représente le module le plus élevé dans la hiérarchie de notre design). Allez dans l'onglet **Sources** de l'espace **Block Design** et sélectionnez votre système (block design). Avec le bouton droit de la souris cliquez sur **Create HDL Wrapper**.



6.2- Assurez-vous que l'option **Let Vivado manage wrapper and auto-update** est choisie puis cliquez sur **OK**.

6.3- Ajout d'un fichier de contraintes

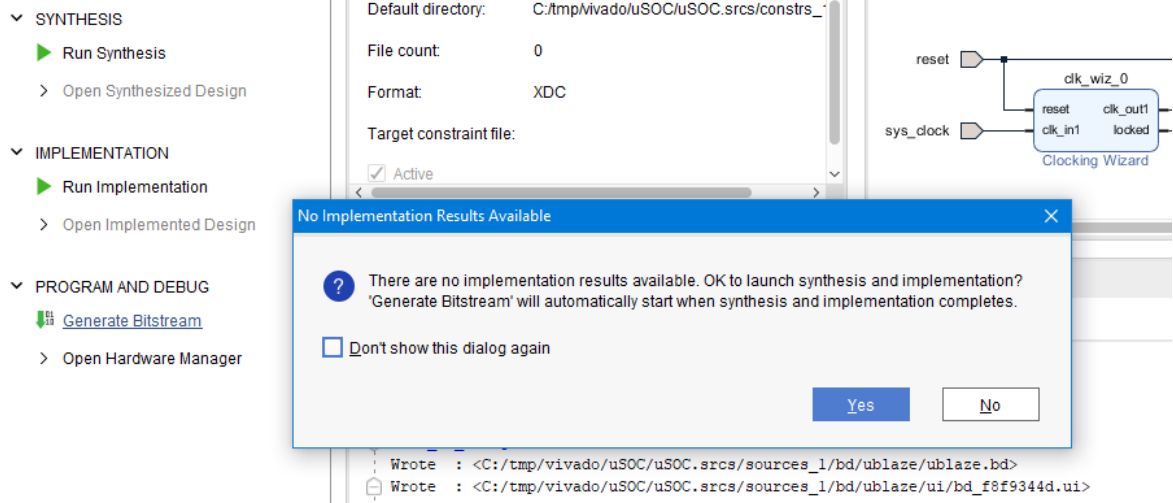
Dans l'espace **BLOCK DESIGN**, sélectionnez l'onglet **Sources**. Ajoutez au projet le fichier de contraintes *basys3_master.xdc* en sélectionnant la rubrique **Constraints**. L'objectif de cette étape est d'associer aux ports d'entrées sorties du toplevel des broches du composant FPGA.



6.4- Cliquez sur le fichier *basys3_master.xdc* pour l'éditer afin d'adapter le nom des broches aux ports d'entrées-sorties du design toplevel. Cette étape termine la conception hardware du système *Microblaze* et autorise la génération du fichier de configuration du FPGA → **"bitstream file"**.

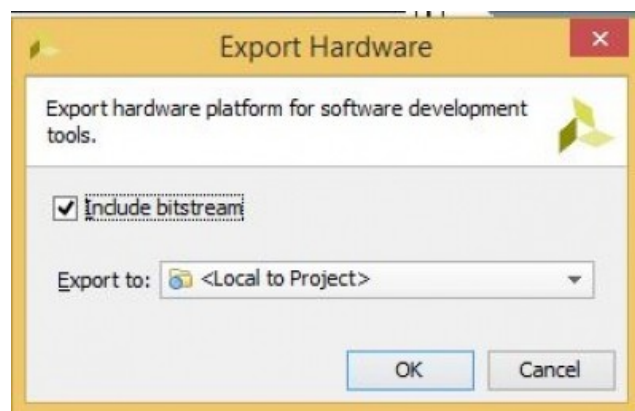
7. Génération du fichier de configuration "bitstream"

7.1- Cliquez sur le menu **Generate bitstream**



8. Préparation de l'environnement de développement software

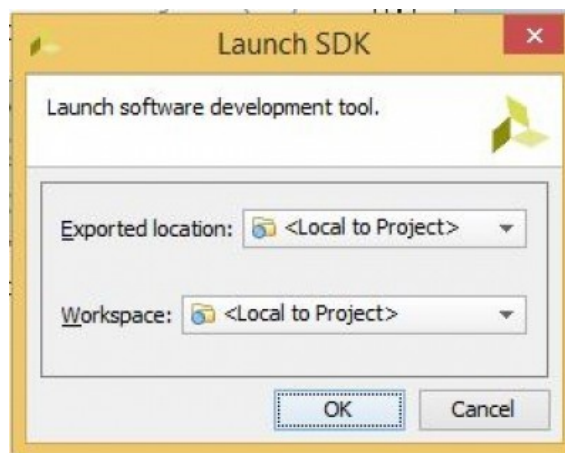
8.1- Dans la barre des menus, cliquez sur **File** et sélectionner **Export**→**Export Hardware**. Cette opération a pour but d'exporter les propriétés de notre système hardware vers l'environnement de développement logiciel **Vivado SDK**.



Un nouveau dossier est alors crée, "*nom_projet.sdk*", et servira dans le développement d'applications software.

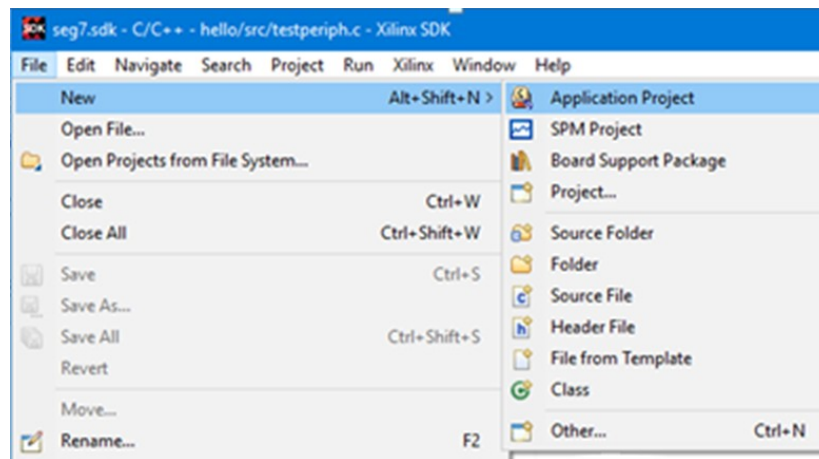
8.2- Dans la barre des menus, cliquez sur **File** et sélectionner **Launch SDK**.

Laissez les choix proposés par défaut et cliquez sur **OK**.

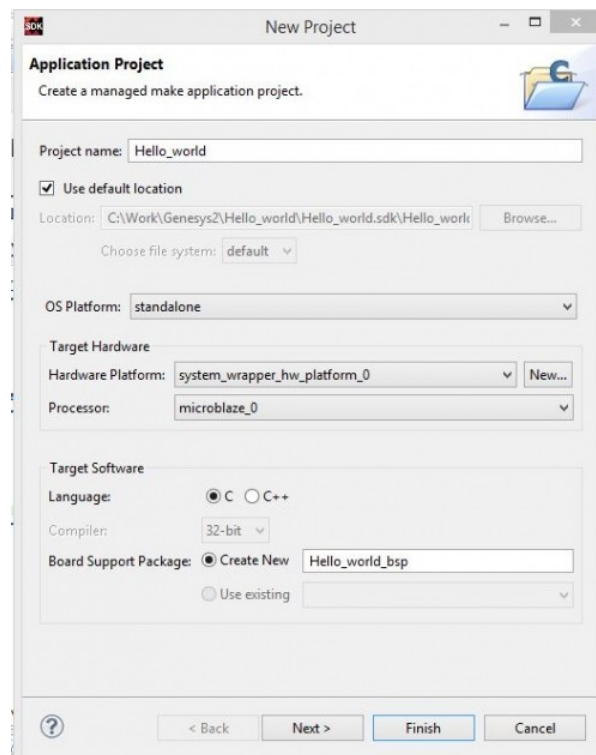


9. Développement d'une nouvelle application software dans SDK

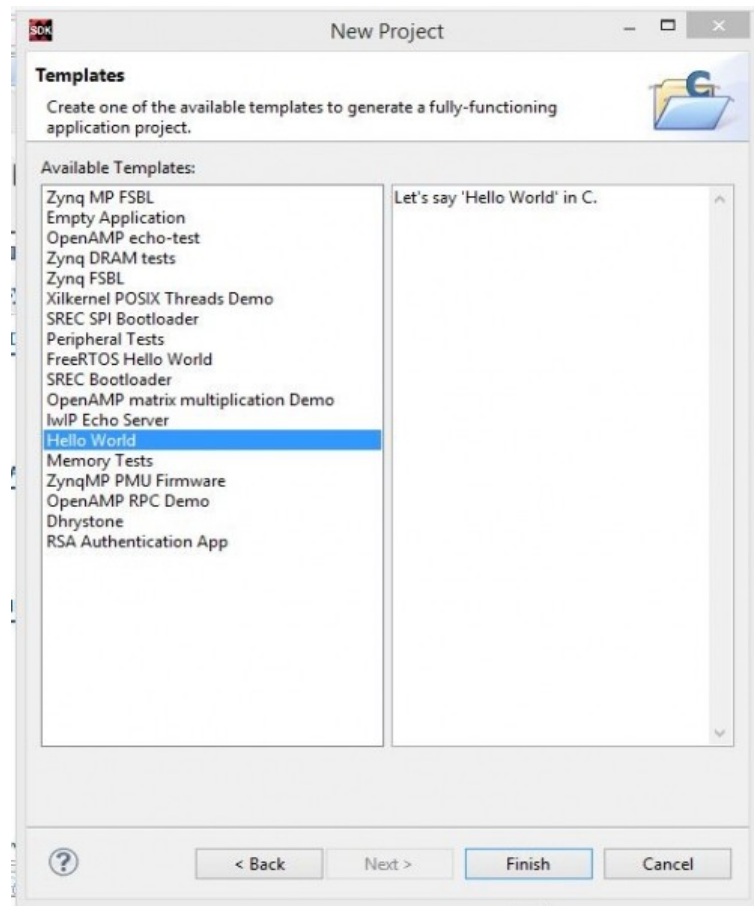
9.1- Une fois SDK lancé, dans la barre des menus, cliquez sur **File** et sélectionnez **new** → **Application Project**.



9.2- Donnez un nom à votre projet et cliquez sur **Next**.



9.3- Sélectionnez un modèle de projet, **Hello World** par exemple, puis cliquez sur **Finish**.



Vous allez constater la création de deux nouveaux dossiers dans l'arborescence du projet software (voir **Project Explorer**). Le premier dossier, **Hello_world**, contient les fichiers sources du projet (*.c et *.h). Le second contient les propriétés du système hardware exportées à partir de Vivado **Board Support Folder**.

9.4- Editez le fichier source ajouté par le template **Hello_world** en cliquant sur **helloworld.c** dans le dossier **src**. Le programme principale **main** contient le message qui sera affiché dans la console à l'exécution de l'application.

```
* Copyright (C) 2009 - 2014 Xilinx, Inc. All rights reserved.

/*
 * helloworld.c: simple test application
 *
 * This application configures UART 16550 to baud rate 9600.
 * PS7 UART (Zynq) is not initialized by this application, since
 * bootrom/bsp configures it to baud rate 115200
 *
 * -----
 * | UART TYPE   BAUD RATE |
 * -----
 * | uartns550   9600      |
 * | uartlite    Configurable only in HW design
 * | ps7_uart    115200 (configured by bootrom/bsp)
 */

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

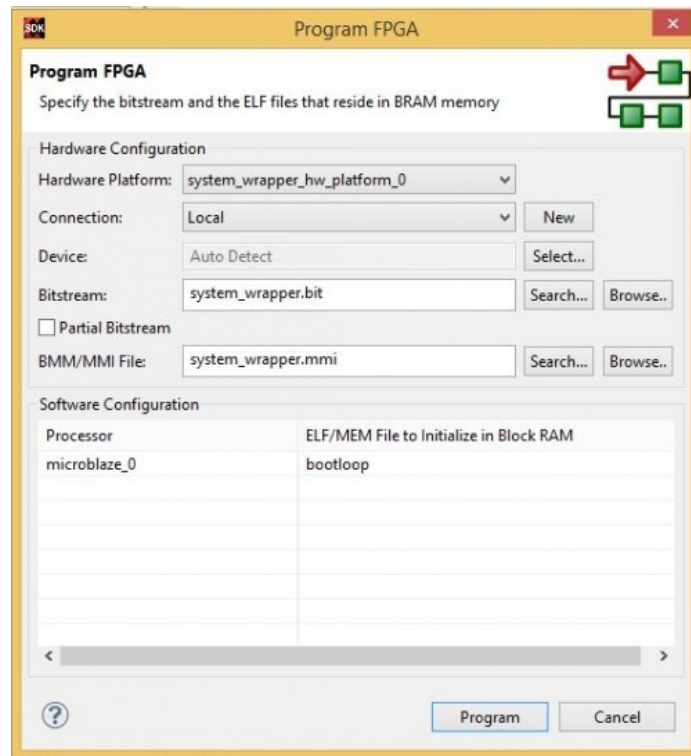
int main()
{
    init_platform();

    xil_printf(" Hello World\n\r");

    cleanup_platform();
    return 0;
}
```

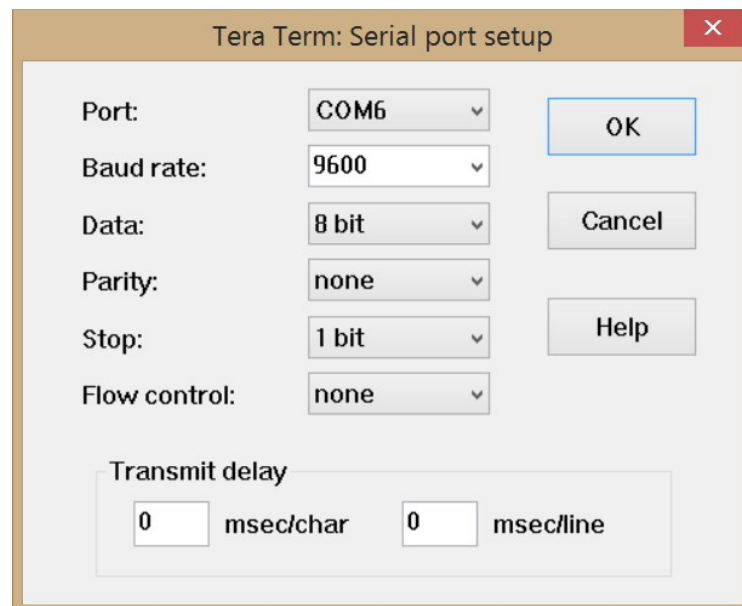
10. Chargement du bitstream dans le FPGA

10.1- Assurez-vous que le kit est sous tension avant de cliquer sur l'icône  **Program FPGA**.



11. Configurer la console **UART Terminal**

11.1- Configurer l'utilitaire qui servira **d'hyperterminal** pour écouter l'UART de Microblaze. Deux possibilités s'offrent à vous : **Tera Term** (à installer) et le **SDK Terminal** de Vivado (de préférence). La configuration de Tera Term ou d'un autre utilitaire du même type, **Putty** est plus accessible, doit se faire selon les propriétés suivantes :

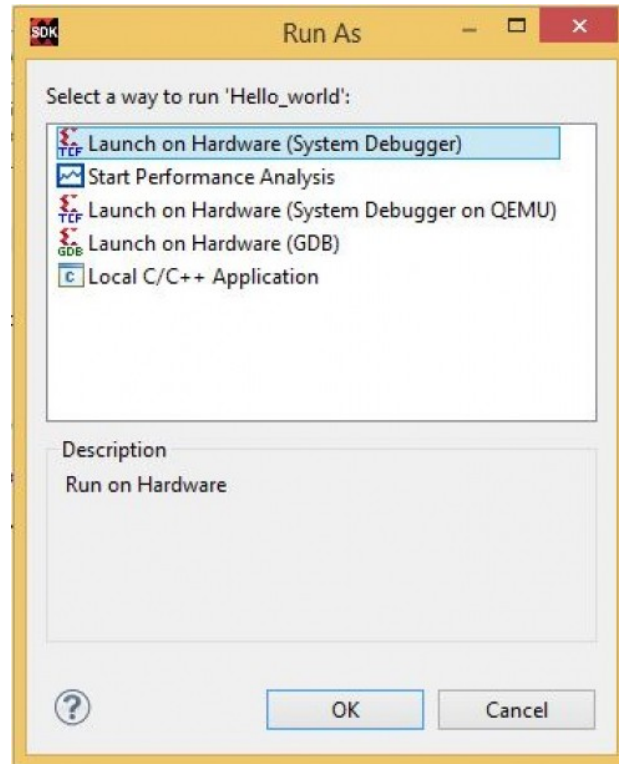


Attention : Le numéro du port effectif est à vérifier dans le panneau de configuration.

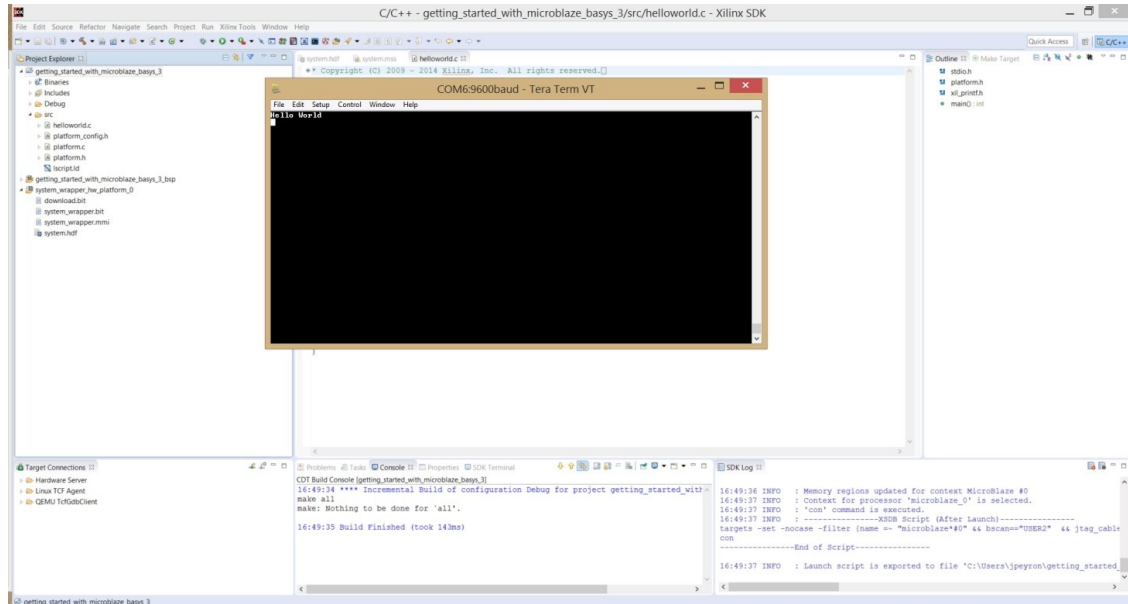
12. Exécution du programme par Microblaze

12.1- Dans SDK, sélectionnez le projet Hello_world puis cliquez sur le menu Run As ...

Dans la fenêtre qui s'ouvre, sélectionnez **Launch on Hardware (System Debugger)** puis cliquez sur **OK**.



12.2- Votre programme s'exécute sur *Microblaze* et vous devez voir le message "Hello World" affiché dans la console de sortie.



Vous venez de réaliser un système sur puce dans un FPGA.

⇒ Bravo, si c'est une première pour vous.

A faire à la suite du tutoriel :

- 1) Dans Vivado, ajoutez des GPIOs pour connecter 16 LEDs et 16 SWITCHES.
- 2) Dans SDK, écrivez des fonctions pour :
 - Allumer une LED sur deux,
 - Réaliser un chenillard simple,
 - Afficher sur les LEDs l'état des SWITCHES